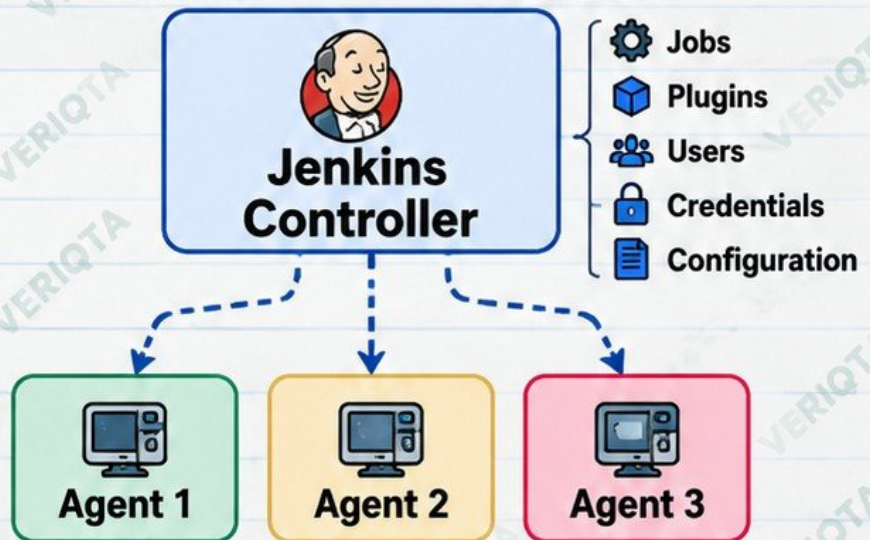


Jenkins

Administrator's Guide



- ✓ Installation
- ✓ Configuration
- ✓ User Management
- ✓ Credentials
- ✓ Plugins
- ✓ Agents
- ✓ Jobs & Pipelines
- ✓ Security
- ✓ Backups
- ✓ Monitoring
- ✓ Troubleshooting
- ✓ Best Practices



Practical
Examples



Real-World
Scenarios



Security
Best Practices



Troubleshooting
Guide



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Jenkins Administration: The Big Picture

1. What is Jenkins?

Jenkins is an open source automation server used to build, test and deploy software. It helps teams automate repetitive tasks in the software delivery process.

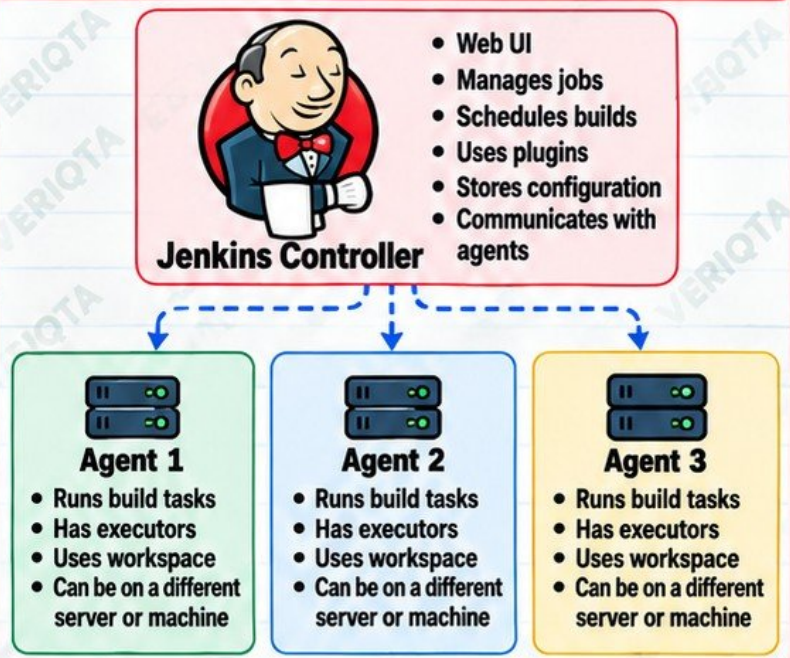
2. Why Do Engineers Use Jenkins?

- ✓ Automatically build and test code
- ✓ Detect problems early
- ✓ Deploy applications faster
- ✓ Integrate with many tools
- ✓ Improve reliability and consistency

3. What Does a Jenkins Administrator Manage?

- Installation and upgrades
- Users and access control
- Plugins and dependencies
- Credentials and security
- Agents and build infrastructure
- Jobs and pipelines
- System configuration
- Backups and monitoring
- Troubleshooting and maintenance

4. Jenkins Architecture (Controller & Agents)



5. Key Jenkins Components

-  **Jobs** – define what Jenkins should do (e.g. build, test, deploy).
-  **Pipelines** – automate the full workflow using a Jenkinsfile.
-  **Plugins** – extend Jenkins with new features and tool integrations.
-  **Credentials** – securely store usernames, passwords, tokens and SSH keys.
-  **Users** – manage access and permissions.
-  **Agents** – run build tasks on separate machines.

6. Where Jenkins Fits in CI/CD



Jenkins automates the build, test and deployment stages in a CI/CD pipeline, helping teams deliver software faster and more reliably.

Junior Engineer Tip



As a Jenkins administrator, your goal is to keep Jenkins stable, secure and available so your team can build, test and deploy their code without issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

How Jenkins Architecture Works

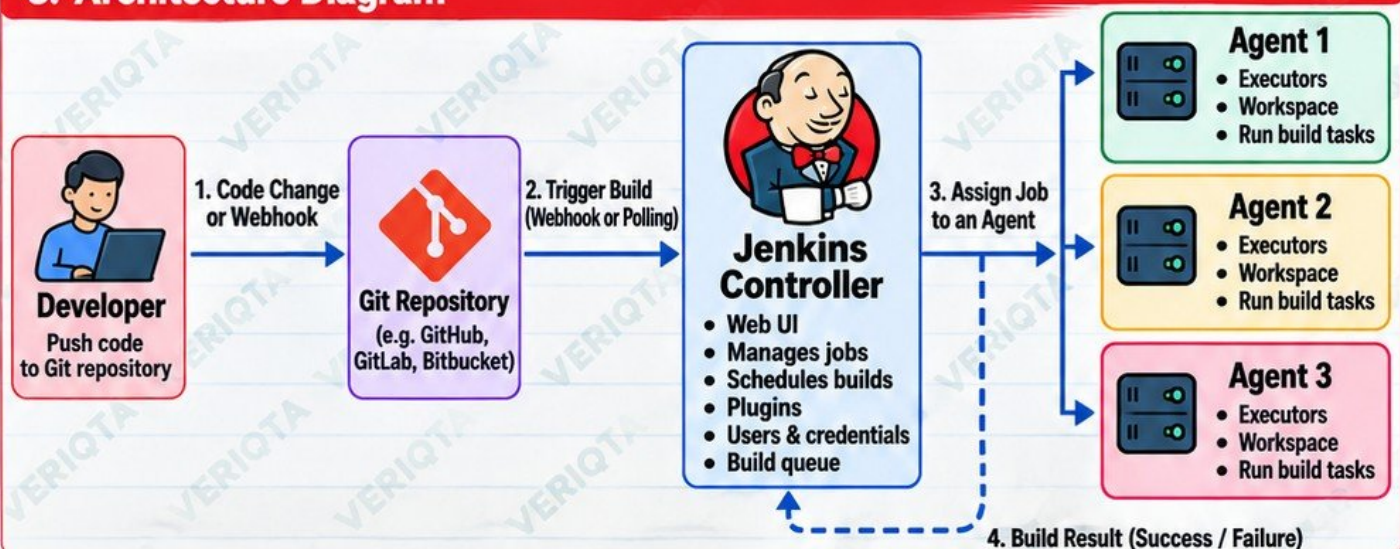
1. Jenkins Architecture Overview

Jenkins follows a controller-agent model. The controller manages jobs, schedules builds and provides the web interface. Agents run the actual build tasks on separate machines.

2. Key Components

- 1 **Controller** – manages jobs, plugins, users and the build queue.
- 2 **Agents** – run build tasks.
- 3 **Executors** – number of concurrent builds an agent can run.
- 4 **Workspace** – directory where the build runs.
- 5 **Jobs / Pipelines** – define what to build, test and deploy.
- 6 **Plugins** – extend Jenkins with new features.

3. Architecture Diagram



4. Build Flow (Step by Step)

- 1 Developer pushes code to a repository.
- 2 Git webhook or polling triggers a Jenkins job.
- 3 Jenkins controller adds the job to the build queue.
- 4 An available agent with free executor picks up the job.
- 5 The agent runs the build in its workspace.
- 6 Build result is sent back to the controller.
- 7 Results are shown in the Jenkins UI and notifications can be sent (e.g. email, Slack).

5. Important Concepts

Controller	Central server that manages everything.
Agent	Runs the actual build tasks.
Executor	Number of builds an agent can run at the same time.
Workspace	Directory on the agent where the job runs.
Build Queue	Holds jobs waiting for an available executor.
Job / Pipeline	Defines the build, test and deployment steps.
Plugins	Add features such as Git, Docker, Kubernetes, etc.
Credentials	Securely store tokens, passwords and SSH keys.
Users	Manage access and permissions.

6. Real World Example



A developer pushes code to GitHub. Jenkins automatically triggers a pipeline. The controller schedules the job on an available agent. The agent runs the build, tests and packaging in its workspace. The result is returned to the controller and shown in the Jenkins UI.

7. Junior Engineer Tip



Use separate agents for heavy builds, keep your controller clean, and monitor the build queue. This improves stability and performance.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Installing Jenkins Correctly

1. Prerequisites (Java Requirement)



- Jenkins requires Java 17 or Java 21 (LTS).
- Check your Java version before installing.
- Use OpenJDK on Linux.
- Jenkins will not start if Java is missing or an unsupported version is installed.

- `java -version`
- `sudo apt update`
- `sudo apt install openjdk-17-jdk -y`

2. Install Jenkins on Linux (Ubuntu/Debian)

- 1 Add the Jenkins repository.
- 2 Install Jenkins.
- 3 Start and enable the service.

```
curl -fsSL https://pkg.jenkins.io/debian-stable/
jenkins.io-2023.key | sudo tee /usr/share/keyrings/
jenkins-keyring.asc > /dev/null

echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | \
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins -y
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

3. Jenkins Service Management

- `sudo systemctl status jenkins` Check service status
- `sudo systemctl start jenkins` Start Jenkins
- `sudo systemctl stop jenkins` Stop Jenkins
- `sudo systemctl restart jenkins` Restart Jenkins
- `sudo systemctl enable jenkins` Enable on boot

4. Initial Setup (Unlock Jenkins)

- 1 Open Jenkins in your browser:
`http://<server-ip>:8080`
- 2 Get the initial admin password:
`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
- 3 Enter the password in the Jenkins web UI.
- 4 Install suggested plugins or choose manually.

5. Create Administrator Account

- After unlocking Jenkins, you will be prompted to create an administrator account.
- Use a strong password.
- This account has full access, so keep it secure.



6. Verify the Installation

- Open your browser and go to:
`http://<server-ip>:8080`
- You should see the Jenkins dashboard.
- Check the version:
`sudo systemctl status jenkins`
Or in the Jenkins UI (bottom right).

7. Important Jenkins Paths

<code>/var/lib/jenkins</code>	Jenkins home directory (jobs, config, plugins)
<code>/etc/default/jenkins</code>	Jenkins service configuration
<code>/var/log/jenkins</code>	Jenkins log files
<code>/usr/share/jenkins</code>	Jenkins application files
<code>/etc/apt/sources.list.d/jenkins.list</code>	Jenkins repository file
<code>/usr/bin/jenkins</code>	Jenkins executable
<code>/var/lib/jenkins/secrets/</code>	Stores initial admin password and secrets

8. Common Issues & Fixes

- ✗ Jenkins not starting → Check Java version and logs
`sudo journalctl -u jenkins`
- ✗ Port 8080 in use → Change port in `/etc/default/jenkins`
- ✗ Cannot access Jenkins UI → Check firewall: `sudo ufw allow 8080`
- ✗ Permission errors → Ensure correct user and file permissions
- ✗ Wrong Java version → Install Java 17 or 21 (LTS)



Junior Engineer Tip: Always use LTS versions of Jenkins and Java. Keep your Jenkins installation updated and secure, especially in production environments.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Understanding JENKINS_HOME

1. What is JENKINS_HOME?

JENKINS_HOME is the main directory where Jenkins stores **all its data**. It contains jobs, configuration, plugins, build history, users and other important files.

2. Default Location



`/var/lib/jenkins`

- This is the default location on most Linux systems.
- The location can be changed during installation using the **JENKINS_HOME** environment variable.
- Always confirm the location on your server.

3. What Jenkins Stores inside JENKINS_HOME

`/var/lib/jenkins`

	<code>jobs</code>	Each job has its own folder (configuration, builds, etc).
	<code>plugins</code>	Installed plugins.
	<code>secrets</code>	Encrypted credentials and sensitive data.
	<code>nodes</code>	Agent configurations.
	<code>logs</code>	Jenkins log files.
	<code>workspace</code>	Build workspaces.
	<code>users</code>	User accounts and settings.
	<code>config.xml</code>	Main Jenkins configuration file.
	<code>credentials.xml</code>	Stored credentials (encrypted).
	<code>hudson.tasks.Maven.xml</code>	Tool configurations (example).
	<code>.jenkins.install.UpgradeWizard.state</code>	Setup wizard state.
	<code>war</code>	Jenkins application files.
	<code>updates</code>	Update center data.
	<code>builds (inside each job)</code>	Build history and artifacts.

4. Key Directories Explained

<code>jobs/</code>	→ Stores each job's configuration, builds and workspace.
<code>plugins/</code>	→ Installed plugin files.
<code>secrets/</code>	→ Encrypted secrets and credentials.
<code>nodes/</code>	→ Agent configurations.
<code>users/</code>	→ User accounts and permissions.
<code>workspace/</code>	→ Temporary directory for build execution.
<code>logs/</code>	→ Jenkins log files.
<code>config.xml</code>	→ Main Jenkins system configuration.

5. Why JENKINS_HOME Matters

- ✓ Contains all Jenkins configuration and data.
- ✓ Holds job definitions and build history.
- ✓ Stores installed plugins and tool configurations.
- ✓ Contains encrypted credentials.
- ✓ You must back it up regularly.
- ✓ If it is lost or corrupted, your Jenkins data is lost.

6. What You Should NOT Do

- ✗ Do not manually edit files inside **JENKINS_HOME** unless you understand the impact.
- ✗ Do not delete files or folders.
- ✗ Do not copy **JENKINS_HOME** while Jenkins is running.
- ✗ Do not store unencrypted sensitive information.
- ✗ Do not change permissions randomly.

7. Best Practices

- ✓ Take regular backups of **JENKINS_HOME**.
- ✓ Stop Jenkins before making manual changes.
- ✓ Monitor disk space (build history can grow quickly).
- ✓ Keep plugins and Jenkins updated.
- ✓ Use proper file permissions (owned by jenkins user).
- ✓ Test restore process periodically.

8. Useful Commands

Check Jenkins home location:

```
echo $JENKINS_HOME
```

List contents:

```
ls -l /var/lib/jenkins
```

Check disk usage:

```
du -sh /var/lib/jenkins
```

Find large files:

```
du -ah /var/lib/jenkins | sort -rh | head -10
```



Junior Engineer Tip: Always keep a backup of **JENKINS_HOME** in a secure location. It contains everything you need to recover your Jenkins server.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Managing Jenkins as a Linux Service

1. What is a Jenkins Service?

- Jenkins runs as a **Linux service (systemd)** on the server.
- The service manages the Jenkins process.
- It starts Jenkins automatically at boot.
- You can start, stop, restart and check the status using **systemctl**.

2. Check Service Status

```
sudo systemctl status jenkins
```

- Shows if Jenkins is running or stopped.
- Displays the service status, PID and recent logs.
- Look for **"active (running)"** when Jenkins is healthy.

3. Start, Stop, Restart and Enable

```
sudo systemctl start jenkins
```

Start Jenkins

```
sudo systemctl stop jenkins
```

Stop Jenkins

```
sudo systemctl restart jenkins
```

Restart Jenkins

```
sudo systemctl enable jenkins
```

Enable on boot

```
sudo systemctl disable jenkins
```

Disable on boot

4. View Service Logs (journalctl)

```
sudo journalctl -u jenkins -f
```

- Shows real-time logs for the Jenkins service.
- Press **CTRL + C** to stop viewing.
- Useful for troubleshooting startup issues.
- Look for error messages at the end of the log.

5. Verify Jenkins is Working

- Check service status:

```
sudo systemctl status jenkins
```
- Open Jenkins in your browser:

```
http://<server-ip>:8080
```
- You should see the Jenkins login page.
- If the page loads, Jenkins is running correctly.



6. Common Service Status Outputs

● active (running)

- Jenkins is running.
- Service is healthy.

● inactive (dead)

- Jenkins is stopped.
- Start the service.

● failed

- Jenkins failed to start.
- Check logs with:

```
sudo journalctl -u jenkins
```

7. Troubleshooting: Jenkins Will Not Start

Possible Cause	What to Check	Fix / Command
Port 8080 in use	Check if another app is using the port	<pre>sudo lsof -i :8080</pre> <pre>sudo kill <PID></pre>
Java not installed or wrong version	Check Java version	<pre>java -version</pre> Install Java 17 or 21 (LTS)
Permission issues	Check ownership of JENKINS_HOME	<pre>sudo chown -R jenkins:jenkins /var/lib/jenkins</pre>
Corrupted config or plugins	Check service logs	<pre>sudo journalctl -u jenkins</pre> Review recent changes
Out of disk space	Check disk usage	<pre>df -h</pre>

8. Service Health Checks

- ✓ Service is active (running).
- ✓ Jenkins UI is accessible on port 8080.
- ✓ No repeated errors in **journalctl** logs.
- ✓ Adequate disk space for **JENKINS_HOME**.
- ✓ Correct Java version (17 or 21 LTS).
- ✓ Required permissions are correct.
- ✓ Plugins and configuration load without errors.

⚠ Junior Engineer Tip



Always check the **journalctl** logs first when Jenkins fails to start. The logs usually show the exact reason.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Users, Roles & Access Control

1. Authentication vs Authorization



Authentication

- Verifies who you are.
- Uses username/password, LDAP, SSO, or other methods.
- After authentication, you can log in to Jenkins.



Authorization

- Decides what you can do.
- Controls permissions after you are logged in.
- Even an authenticated user may not have permission to perform certain actions.

2. Jenkins Users

- Jenkins can manage its own users or integrate with external systems (LDAP, Active Directory, SSO).
- Each user has a unique identity.
- Users can belong to groups (when using role-based access plugins).
- Always use individual accounts, not shared accounts.
- Remove inactive users.



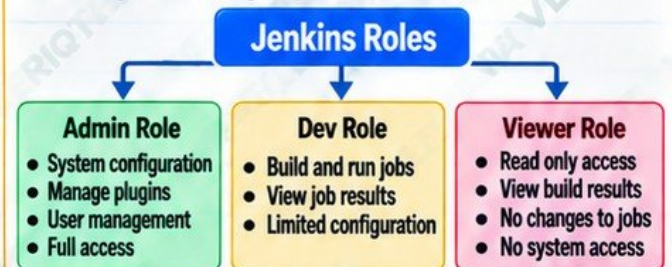
3. Permissions in Jenkins

- Permissions control what a user can do.
- Examples:

Permission	What It Allows
Overall/Read	View Jenkins and job information.
Overall/Build	Trigger a build.
Job/Configure	Configure a job.
Job/Read	View a job.
Job/Delete	Delete a job.
Overall/Administer	Full control of Jenkins (very sensitive).

4. Role-Based Access Control (RBAC)

- Use the Role-Based Authorization Strategy plugin for better access management.
- Create roles based on responsibilities.
- Assign users or groups to specific roles.



5. Least Privilege Principle

- ✓ Give users only the permissions they need.
- ✓ Avoid giving Overall/Administer unless required.
- ✓ Use role-based access instead of giving everyone full access.
- ✓ Regularly review user permissions.
- ✓ Remove unused permissions.
- ✓ Use groups to manage access more easily.



6. Administrator Access

- Administrator account has full control.
- Use a strong password.
- Limit the number of administrator accounts.
- Use multi-factor authentication (MFA) if possible.
- Avoid using the administrator account for daily activities.
- Keep an audit of administrator actions (check logs).

7. Why Unrestricted Access is Dangerous

- ✗ Users can delete jobs or configurations.
- ✗ Sensitive credentials can be exposed.
- ✗ Malicious or accidental changes can break the entire Jenkins server.
- ✗ Attackers can install unauthorized plugins.
- ✗ Build pipelines can be modified to execute harmful code.
- ✗ Loss of audit trail and accountability.



8. Best Practices

- ✓ Use role-based access control (RBAC).
- ✓ Follow the least privilege principle.
- ✓ Integrate with LDAP/SSO for centralized user management.
- ✓ Regularly audit users and permissions.
- ✓ Remove inactive accounts.
- ✓ Use MFA for administrator accounts.
- ✓ Monitor Jenkins logs for suspicious activities.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Jenkins Credentials Management

1. What are Jenkins Credentials?

- Jenkins credentials store sensitive information used by jobs and pipelines.
- They are used to **authenticate** with external systems like Git, Docker, cloud providers, etc.
- Credentials are **encrypted** and stored in **JENKINS_HOME**.
- Using credentials is **more secure** than hardcoding secrets in your pipelines.

2. Credential Types in Jenkins

Credential Type	Use Case
Username with password	For systems that require a username and password (e.g. Git, servers).
SSH Username with private key	For SSH access to servers or Git repositories.
Secret text	For tokens, API keys or any text secret.
Secret file	For files like kubeconfig, certificates, etc.
Certificate	For client certificates and keys.
AWS credentials	For AWS access (Access Key and Secret).
Docker registry	For Docker registry authentication.
GitHub App / Token	For GitHub authentication.

3. Creating Credentials in Jenkins

- Go to **Manage Jenkins** → **Credentials**.
- Choose the appropriate scope (Global or Folder).
- Click Add Credentials.
- Select the credential type.
- Enter the required details (ID, description, secret).
- Click **OK**.



4. Credential Scope

- Global:** Available to all jobs and folders.
- Folder:** Available only to jobs inside a specific folder.
- System:** Used by Jenkins internally.
- User:** Available only to a specific user.



Use the least scope required to follow the principle of least privilege.

5. Using Credentials in a Pipeline

Example 1: Using a Username and Password

```
pipeline {
  agent any
  stages {
    stage('Deploy') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'git-cred',
          usernameVariable: 'GIT_USER',
          passwordVariable: 'GIT_PASS'
        )]) {
          sh 'git clone https://${GIT_USER}:${GIT_PASS}@github.com/example/repo.git'
        }
      }
    }
  }
}
```

Example 2: Using an SSH Key

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        withCredentials([sshUserPrivateKey(
          credentialsId: 'ssh-cred',
          keyFileVariable: 'SSH_KEY'
        )]) {
          sh 'git clone git@github.com:example/repo.git'
        }
      }
    }
  }
}
```

6. Important Tips

- ✓ Always use credentials instead of hardcoding secrets.
- ✓ Use descriptive credential IDs (e.g. github-prod-token).
- ✓ Limit the scope to only what is required.
- ✓ Rotate credentials regularly.
- ✓ Use folder-level credentials for specific projects.
- ✓ Audit and remove unused credentials.
- ✓ Avoid giving administrator access unnecessarily.



7. What NOT to Do

- ✗ Do not hardcode passwords, tokens or keys in Jenkinsfiles.
- ✗ Do not share credentials between unrelated projects.
- ✗ Do not use global credentials when folder scope is enough.
- ✗ Do not give unnecessary administrator access.
- ✗ Do not store secrets in Git repositories.
- ✗ Do not expose credentials in build logs.
- ✗ Do not use weak or easily guessable credentials.



Junior Engineer Tip: Use meaningful credential IDs, restrict the scope, and never hardcode secrets in your pipelines or scripts. Treat credentials like sensitive production data.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Jenkins Plugins Without Breaking Jenkins

1. What are Jenkins Plugins?

- Plugins **extend** Jenkins with new features.
- They add **integrations**, tools, UI improvements and security features.
- Plugins are developed by the Jenkins community and third parties.
- While plugins make Jenkins powerful, they can also cause issues if not managed properly.



2. Common Plugin Use Cases

Plugin Category	Examples
Source Control	Git, GitHub, GitLab, Bitbucket
Build Tools	Maven, Gradle, Ant
Cloud / Infrastructure	Docker, Kubernetes, AWS, Azure
Notifications	Email, Slack, Microsoft Teams
Security	LDAP, SAML, Role-based access
Pipeline & Workflow	Pipeline, Blue Ocean, Multibranch
Utilities	Credentials, Parameterized Trigger

3. Plugin Dependencies

- Many plugins depend on **other plugins**.
- If a required plugin is missing, Jenkins may fail to start or the feature may not work.
- Always check the Required **Plugins** section before installing.



4. Compatibility

- Plugins must be compatible with your Jenkins version (especially LTS).
- Check the plugin page for the required Jenkins core version.
- Using incompatible plugins can cause errors or prevent Jenkins from starting.



5. Installing and Updating Plugins

- 1 Go to **Manage Jenkins** → **Plugins**.
- 2 Use the **Available** tab to search for plugins.
- 3 Select the plugin and click **Install** (restart if required).
- 4 To update plugins, go to the **Updates** tab.



6. Security Advisories

- Some plugins may have known security vulnerabilities.
- Check the **Security** tab in the Plugin Manager.
- Keep plugins updated to protect your Jenkins server.
- Remove or disable plugins with critical security issues.



7. Testing Plugin Upgrades

- Always test plugin updates in a non-production environment first.
- Verify that your jobs and pipelines still work.
- Check for breaking changes in the release notes.
- Keep a backup of **JENKINS_HOME** before major updates.



8. Removing Unused Plugins

- Regularly review installed plugins.
- Remove plugins that are not used.
- Unused plugins increase security risk, slow down Jenkins and can cause compatibility issues during upgrades.



9. Useful Commands and Checks

# List installed plugins <code>ls \$JENKINS_HOME/plugins</code>	View installed plugins.
# Check plugin version (example) <code>grep -h 'Plugin-Version'</code> <code>\$JENKINS_HOME/plugins/*/META-INF/MANIFEST.MF</code>	Check a specific plugin version.
# View Jenkins logs after plugin update <code>sudo journalctl -u jenkins -f</code>	Monitor logs for errors.

10. ⚙️ Best Practices

- ✓ Install only required plugins.
- ✓ Keep plugins updated.
- ✓ Check compatibility before installing.
- ✓ Review security advisories regularly.
- ✓ Test upgrades in a staging environment.
- ✓ Remove unused plugins.
- ✓ Keep a backup of **JENKINS_HOME**.
- ✓ Document installed plugins and their purpose.
- ✓ Avoid uncontrolled plugin growth.



Junior Engineer Tip: Less plugins = more stable, secure and easier to maintain Jenkins.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Creating & Managing Jenkins Agents

1. Why Jenkins Agents Exist?

- Agents allow Jenkins to run builds on separate machines.
- They help **distribute workloads** and keep the controller lightweight.
- Agents can run on Linux, Windows, virtual machines or cloud instances.
- You can have multiple agents for different tasks and environments.



2. Controller vs Agent Responsibilities

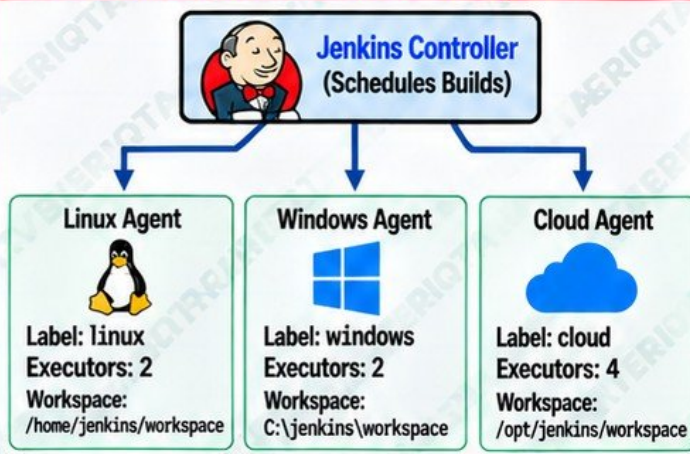
Jenkins Controller

- Manages configuration, users, jobs and plugins.
- Schedules builds on agents.
- Stores build history and metadata.
- Provides the web UI.
- Does not run heavy build work (should be avoided).

Jenkins Agent

- Executes build jobs and pipeline steps.
- Runs on a separate machine.
- Uses executors to run tasks.
- Has its own workspace.
- Connects to the controller over a secure channel.
- Can be added or removed as needed.

3. Jenkins Architecture with Agents



4. Labels and Executors

Labels

- Labels help you assign jobs to specific agents.
- You can use labels based on OS, tools or environment.
- Example labels:

linux windows
docker kubernetes
prod staging

Executors

- Executors define how many builds can run on an agent at the same time.
- Each agent can have multiple executors.
- If all executors are in use, new builds wait in the queue.



Example:
Executors: 2
→ Can run 2 builds at the same time.

5. Agent Workspaces

- Each agent has its own workspace directory where the source code and build files are stored.
- The workspace is isolated per agent.
- Default locations:

- Linux: /home/jenkins/workspace
- Windows: C:\jenkins\workspace
- Cloud: /opt/jenkins/workspace



6. Agent Connectivity

- Agents connect to the controller using a secure channel.
- Common connection methods:

SSH (recommended for Linux agents)

JNLP / WebSocket

Launch agents using cloud plugins (AWS, Azure, Kubernetes)



7. Managing Agents in Jenkins

- Go to Manage Jenkins → **Nodes (Agents)**.
- Click New Node.
- Choose the agent type (Permanent, SSH, etc).
- Configure name, labels, executors and connection details.
- Save and launch the agent.



8. Distributing Workloads

- Use multiple agents for parallel builds.
- Assign labels to run specific jobs on specific agents.
- Separate environments (dev, staging, prod).
- Use dedicated agents for heavy tasks like Docker builds or integration tests.
- Scale agents using cloud providers when demand increases.

9. Best Practices

- Use agents instead of running builds on the controller.
- Use meaningful labels.
- Set the correct number of executors.
- Keep agents updated.
- Monitor agent health and connectivity.
- Remove unused agents.
- Use cloud agents for scalability.

Junior Engineer Tip: Use agents to keep your Jenkins controller stable and to run builds in parallel for faster delivery.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

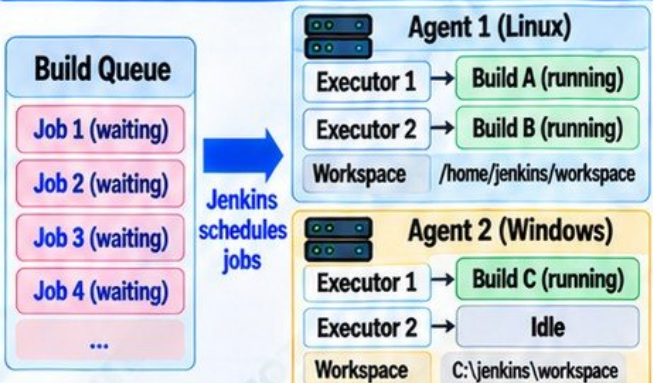
Executors, Build Queue & Workspaces

1. How Jenkins Schedules Work

- When a job is triggered, Jenkins adds it to the **build queue**.
- Jenkins looks for an **available agent** with free **executors** that matches the job requirements (labels, node type, etc).
- The job is assigned to an executor on an agent.
- If no executors are available, the job waits in the queue until one becomes free.



2. Key Components



3. Executors vs Agents

Executors

- Run the actual build jobs in parallel.
- Each executor can run one build at a time.
- An agent can have multiple executors.
- Executors determine how many concurrent builds can run on an agent.



Agents

- Are machines (physical, virtual or cloud) that run builds.
- Can be Linux, Windows or container-based.
- Have one or more executors.
- Can be assigned labels to run specific types of jobs.



4. Build Queue

- Holds jobs that are waiting for an available executor.
- Jobs stay in the queue if:

- All matching executors are busy.
- No agent matches the required label.
- The agent is offline.
- There are more builds than available executors.



- You can view the queue in Jenkins under:

Manage Jenkins → System Information → Build Queue

5. Workspaces

- Each job runs in a workspace on the agent.
- The workspace stores the source code, build files and temporary files.
- Default workspace locations:



Linux: /home/jenkins/workspace
Windows: C:\jenkins\workspace
Docker/Container: /var/jenkins_home/workspace

- Each job normally has its own folder inside the workspace.

6. Concurrency and Capacity

- More executors = more parallel builds.
- Monitor queue length to understand if you need more agents or executors.
- Use labels to distribute different types of jobs (build, test, deploy) to the right agents.
- Avoid running CPU or memory heavy jobs on the same agent if they affect other builds.
- Scale out by adding more agents when the queue is always busy.



7. Common Issues and Causes

Issue	Possible Cause
Job stuck in queue	No available executors, no matching label, or agent offline.
Job never starts	Agent is disconnected or lacks required label.
Workspace not found	Workspace deleted or agent reconfigured.
Slow builds	Not enough executors or insufficient resources (CPU/RAM/disk).
Builds interfering	Jobs sharing the same workspace or running on the same agent with limited resources.

8. Troubleshooting Tips

- ✓ Check the build queue to see if jobs are waiting.
- ✓ Verify agent status (online/offline).
- ✓ Check the number of executors on each agent.
- ✓ Ensure the job is using the correct label.
- ✓ Look at agent logs for errors.
- ✓ Increase executors or add more agents if the queue is always busy.
- ✓ Clean up old workspaces if disk space is low.
- ✓ Monitor system resources (CPU, memory, disk).



Junior Engineer Tip: If your builds are always in the queue, you need more executors or more agents.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Jobs, Pipelines & Jenkinsfiles

1. Types of Jenkins Jobs

Freestyle Job

- Traditional job type.
- Configured through the UI.
- Good for simple build tasks.
- Not ideal for complex CI/CD.
- Configuration is stored in Jenkins (not in Git).
- Harder to maintain and share between teams.



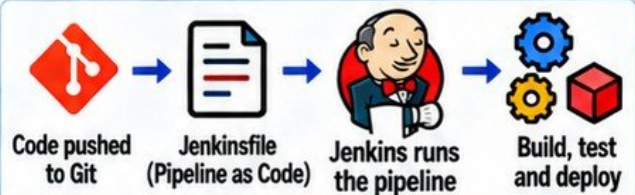
Pipeline Job

- Defined using a Jenkinsfile (Pipeline as Code).
- Supports complex CI/CD.
- Stored in source control (e.g. Git).
- Easier to maintain, review and version.
- Recommended for modern DevOps workflows.



2. Pipeline as Code

- Pipeline is defined in a text file (Jenkinsfile).
- Stored in version control (e.g. Git).
- Can be reviewed, updated and tracked like any other code.
- Improves collaboration, consistency and security.
- Supports automated CI/CD from commit to deployment.



3. Declarative vs Scripted Pipeline

Feature	Declarative Pipeline	Scripted Pipeline
Syntax	Simpler and structured	More flexible (Groovy)
Ease of use	Easier for beginners	For advanced use cases
Readability	Clean and easy to read	More complex
Error handling	Built-in structure	Manual handling required
Use case	Recommended for most projects	When you need full control and advanced logic

5. Key Concepts in a Pipeline

Stages	Divide the pipeline into logical steps (e.g. build, test, deploy).
Steps	Actual commands or actions to run (e.g. sh, bat, git, docker).
Agent	Defines where the pipeline will run (e.g. any, specific label).
Environment	Set environment variables for the pipeline.
Post	Actions to run after the pipeline (e.g. success, failure).



4. Jenkinsfile Structure (Declarative)

```

pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/example/repo.git'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
    stage('Deploy') {
      steps {
        sh './deploy.sh'
      }
    }
  }
}
  
```

Annotations:

- Pipeline definition
- Agent (where to run)
- Stages (build steps)
- Steps (commands)

6. Why Use Version-Controlled Pipelines?

- ✓ Configuration is stored in Git (Jenkinsfile).
- ✓ Allows code review and change tracking.
- ✓ Easier to maintain and rollback.
- ✓ Promotes consistency across environments.
- ✓ Supports Infrastructure as Code and GitOps practices.
- ✓ Enables better collaboration between developers and DevOps teams.



7. Best Practices

- ✓ Use Pipeline as Code instead of freestyle jobs.
- ✓ Store Jenkinsfile in your application repository.
- ✓ Keep pipelines simple and modular.
- ✓ Use meaningful stage names.
- ✓ Use parameters and environment variables.
- ✓ Handle failures and notifications (post section).
- ✓ Reuse common pipeline code with shared libraries.
- ✓ Enforce code review for Jenkinsfile changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Connecting Jenkins to Git Repositories

1. Why Integrate with Git?

- Store your code in **Git** (GitHub, GitLab, Bitbucket or self-hosted).
- Automatically **trigger builds** when code changes.
- Keep build configuration in version control (Jenkinsfile).
- Improves collaboration, traceability and consistency.
- Supports modern DevOps workflows.



2. Repository Configuration

- 1 Go to your Jenkins job (Freestyle or Pipeline).
- 2 Configure the **Source Code Management** section.
- 3 Select **Git** and enter your repository URL.
- 4 Choose the branch specification (e.g. main or */*).
- 5 Add credentials (if the repository is private).
- 6 Save the configuration.

Source Code Management

☐ None

☒ Git

Repository URL

Credentials

Branches to build

3. Jenkins Credentials for Git

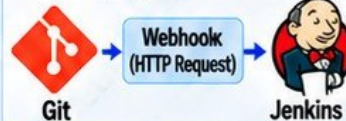
- Use **credentials** for private repositories.
- Common credential types:
 - Username and password
 - SSH private key
 - Personal access token (PAT)
 - GitHub App (advanced)
- Store credentials securely in Jenkins.
- Give the credential a clear ID and description.
- Use least privilege access (read-only when possible).



4. Webhooks vs Polling

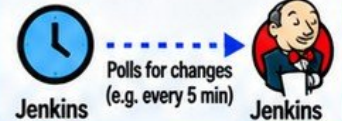
Webhooks (Recommended)

- Git sends a notification to Jenkins when code is pushed.
- Build starts immediately.
- More efficient, real-time.
- Supported by GitHub, GitLab, Bitbucket, etc.



Polling (Alternative)

- Jenkins checks the repository periodically for changes.
- Builds start based on a schedule (e.g. every 5 minutes).
- Useful when webhooks are not available.

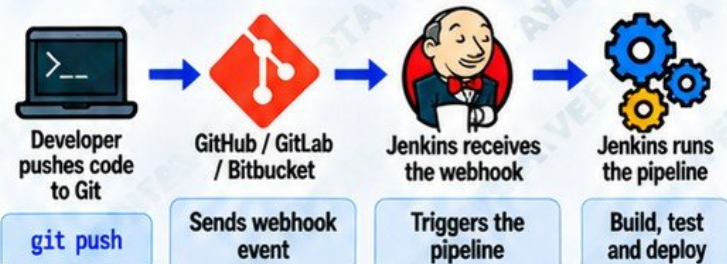


5. Branches and Triggers

- Build a specific branch (e.g. main).
- Build multiple **branches** using wildcards (e.g. */*).
- Use **branch filters** for Pull Requests (if using Multibranch Pipeline).
- Configure triggers:
 - On push (webhook)
 - On schedule (polling)
 - On Pull Request events
 - On tag creation (e.g. v1.0.0)



6. Flow from Git Push to Jenkins Pipeline



7. Example: Pipeline from a Git Repository

```

pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/your-org/your-repo.git'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }
  }
}
  
```

- Jenkinsfile is stored in the Git repository.
- Jenkins checks out the code and runs the pipeline.
- Any change to the Jenkinsfile is version controlled.
- Keeps your build process transparent and repeatable.

8. Best Practices

- ✓ Use webhooks instead of polling when possible.
- ✓ Keep your Jenkinsfile in the repository.
- ✓ Use credentials with least privilege.
- ✓ Build only the required branches.
- ✓ Use meaningful branch and tag naming.
- ✓ Monitor webhook delivery and build triggers.
- ✓ Store sensitive information in Jenkins credentials, not in the repository.
- ✓ Test your configuration with a small change first.
- ✓ Review and clean up unused triggers.
- ✓ Use Multibranch Pipeline for large projects.



Junior Engineer Tip:

Always use webhooks and a Jenkinsfile. It makes your CI/CD pipeline faster, more reliable and easier to maintain.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Managing Jenkins Configuration Safely

1. Jenkins Global Configuration

- Global configuration controls system-wide settings that affect all jobs and users.
- Access it from: **Manage Jenkins** → **System**.
- Common areas: system settings, tools, plugins, security, email, and cloud configurations.
- Changes here affect the entire Jenkins instance.



2. System Settings

- Configure the Jenkins URL.
- Set system messages and administrator contacts.
- Configure email notifications (SMTP).
- Set global build discarder (to manage disk usage).
- Configure proxy settings (if required).
- Adjust security-related settings.

Manage Jenkins → System

Jenkins URL
 System Admin E-mail
 SMTP Server



3. Tools Configuration

- Define and manage tools that jobs will use (installed automatically or provided by the system).
- Common tools: JDK, Maven, Git, Docker, Node.js, Python, etc.
- Tools configured globally can be referenced in pipelines.



Manage Jenkins → Tools

JDK	JDK 17 (/usr/lib/jvm/java-17)
Maven	Maven 3.9.6
Git	Git (system)
Docker	Docker (/usr/bin/docker)

4. Environment Configuration

- Configure global environment variables.
- Set up paths for tools and custom scripts.
- Define environment variables for all jobs or specific folders.
- Use environment variables to keep paths, URLs and credentials consistent.



Manage Jenkins → System → Global properties

☒ Environment variables

Name	Value
JAVA_HOME	/usr/lib/jvm/java-17
MAVEN_HOME	/opt/maven
DOCKER_HOST	unix:///var/run/docker.sock

5. Making Configuration Changes

- Plan the change (know what you want to achieve).
- Document the current configuration (take a backup).
- Make the change in a non-production environment first.
- Test that jobs and pipelines still work.
- Apply the change during a maintenance window (if possible).



6. Testing Changes

- Verify that critical jobs still run successfully.
- Check agent connectivity and tool availability.
- Confirm integrations (Git, email, notifications, etc).
- Monitor system logs for errors after the change.
- Ask a small set of users to test important pipelines.



7. Documenting Administration Decisions

- Keep a record of what was changed, why it was changed, when it was changed, and who made the change.
- Document tool versions, plugins, environment variables and system settings.
- Use a simple change log (Markdown file, wiki or ticketing system).
- Good documentation helps with troubleshooting and future maintenance.



8. What to Avoid

- Do not experiment directly on a production Jenkins server.
- Do not make untested changes that can break running jobs.
- Do not remove plugins or tools without checking dependencies.
- Do not expose sensitive information in global configuration.
- Do not give unnecessary administrator access.
- Do not skip backups before major configuration changes.



Junior Engineer Tip:

Always document, test in a non-production environment and keep backups before making significant configuration changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Jenkins Logs & Troubleshooting

1. Where to Investigate Failures?

- Jenkins system logs (server logs).
- Linux service logs (systemd logs).
- Build console output (job logs).
- Agent logs (node logs).
- Plugin logs (for plugin-related issues).
- System metrics (disk, memory, CPU, network).



2. Jenkins System Logs

- Jenkins stores detailed logs in the logs directory.
- Look for errors, warnings and stack traces.
- Useful for startup issues, plugin errors and general problems.



```
# Jenkins log location (default)
/var/log/jenkins/jenkins.log
# View latest logs
tail -f /var/log/jenkins/jenkins.log
# Search for errors
grep -i error /var/log/jenkins/jenkins.log
```

3. Linux Service Logs

- Use systemd to check Jenkins service logs.
- Helpful when Jenkins will not start or keeps stopping.

```
# View service logs
journalctl -u jenkins -f
# View last 100 lines
journalctl -u jenkins -n 100
# View logs from a specific time
journalctl -u jenkins --since "2024-01-01"
```



4. Build Console Output

- Shows real-time logs for each build.
- Use it to identify pipeline failures, command errors and test results.
- Available from the Jenkins UI → Job → Build → Console Output.

Common things to check:

- Command errors
- Test failures
- Missing dependencies
- Environment variables
- Artifact issues



Console Output

5. Agent Logs

- Check agent logs when builds fail on a specific node.
- Look for connection issues, workspace problems or missing tools.

```
# Example locations (may vary)
/var/log/jenkins/agent.log
/var/log/jenkins/slave-agent.log
# For inbound agents (Java -jar agent.jar)
Check the agent console where it was started.
```



6. Plugin Logs and Errors

- Plugin issues can cause build failures or prevent Jenkins from starting.
- Check the system log for plugin-related errors.
- If a specific plugin is causing problems, you may see stack traces or `ClassNotFoundException` errors.



```
# Search for plugin related errors
grep -i plugin /var/log/jenkins/jenkins.log
# Example error patterns
grep -i "ClassNotFoundException" /var/log/jenkins/jenkins.log
```

7. Troubleshooting Workflow (Symptom → Evidence → Cause → Fix → Verify)

1. Symptom

- Identify the problem.
- What is failing?
- When does it happen?
- Any error message?

2. Evidence

- Check relevant logs (system, build, agent).
- Look for error messages, stack traces or warnings.
- Gather timestamps.

3. Cause

- Identify the root cause based on the evidence.
- Check configuration, plugins, tools, permissions or resources.

4. Fix

- Apply the correct fix.
- Update configuration, install missing tools, fix permissions or resolve resource issues.

5. Verify

- Run the job again.
- Check that the issue is resolved.
- Monitor for recurring problems.

8. Common Issues and Solutions

Symptom	Evidence	Possible Cause	Fix
Jenkins will not start	Service logs	Port in use, config error	Fix config, free port
Build fails immediately	Console output	Missing tool or dependency	Install tool, check path
Agent offline	Agent logs	Network or credentials issue	Check connection, credentials
Plugin errors	System log	Incompatible plugin version	Update or rollback plugin
Out of disk space	System log	Large workspaces or logs	Clean workspaces, rotate logs

9. Best Practices

- ✓ Check the right log for the right issue.
- ✓ Use clear error messages to identify the cause.
- ✓ Keep logs clean and rotate old logs.
- ✓ Monitor disk space, memory and CPU usage.
- ✓ Document common issues and their solutions.
- ✓ Avoid making changes directly on production without testing.
- ✓ Set up alerts for critical failures (service down, agent offline, disk full).



Junior Engineer Tip:

When troubleshooting, always start with the logs. Logs tell you what happened, why it happened and what to fix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Troubleshooting Failed Builds

1. Identify the Type of Problem

- First, decide if the failure is a **Jenkins infrastructure problem** or an **application (build) problem**.
- This helps you focus on the right logs and solution.



Jenkins Infrastructure Problems

- Jenkins not starting
- Agent offline
- Missing tools
- Credentials issues
- Network or permission problems
- Disk space or workspace issues

VS

Application (Build) Problems

- Code compilation errors
- Test failures
- Dependency issues
- Build script errors
- Application configuration issues
- Docker build failures
- Deployment failures

2. Check the Console Output

- The console output shows the exact steps that ran.
- Read from top to bottom to find the first error.
- Look for keywords like: **ERROR**, **FAILED**, **Exception**, **Permission denied**, **Not found**, **Timeout**.

Example (Console Output)

```
[Pipeline] sh
+ mvn clean install
[ERROR] Failed to execute goal
[ERROR] Could not resolve dependencies
Caused by: java.net.UnknownHostException:
repo.maven.apache.org
```



- The first error is usually the real problem. Later errors are often just a result of the first one.

3. Common Causes and How to Troubleshoot

Issue	Symptoms / Errors	How to Troubleshoot
Missing tools	command not found	Check tool installation and PATH. Verify tool configuration in Jenkins.
Permissions	Permission denied	Check file permissions, user access and credential privileges.
Credentials	Authentication failed	Verify correct credentials ID and type. Ensure credentials have required access.
Workspace issues	No space left on device	Check disk space and clean workspace. Remove old builds and artifacts.
Network failures	Connection timed out	Check network connectivity, proxy settings and firewall rules.
Agent failures	Agent offline / Not connected	Check agent status, logs and connectivity. Restart the agent if needed.
Application errors	Build or test failed	Check the build logs, code, dependencies and test configuration.

4. Checking Specific Areas



Tools

- Verify required tools are installed.
- Check tool versions and paths.



Permissions

- Ensure Jenkins user has access.
- Check file and folder permissions.



Credentials

- Confirm correct credentials ID.
- Test credentials outside the build.



Workspace

- Check workspace path and disk space.
- Clean or delete corrupted workspace.



Network

- Test connectivity to external services (Git, artifact repositories, Docker, etc).
- Check proxy and firewall settings.



Agents

- Verify agent is online and has free executors.
- Check agent logs for errors.

5. Symptom → Evidence → Cause → Fix → Verify

1. Symptom

- What happened?
- When does it happen?
- Any error message?

2. Evidence

- Check console output, system logs, agent logs.
- Look for the first error.
- Gather relevant details.

3. Cause

- Identify the root cause.
- Is it Jenkins, agent, network, tools, credentials or application code?

4. Fix

- Apply the correct fix.
- Update config, install tools, fix permissions, resolve code or network issue.

5. Verify

- Run the build again.
- Check that the issue is resolved.
- Monitor for recurrence.

6. Useful Commands

```
# Check Jenkins service status (Linux)
sudo systemctl status jenkins
# View Jenkins system logs
sudo journalctl -u jenkins -f
# Check disk space
df -h
# Check running processes
ps aux | grep jenkins
# Check network connectivity
ping github.com
# View agent logs (example path)
tail -f /var/log/jenkins/agent.log
```



7. Best Practices

- Read the full console output, not just the last lines.
- Always check the first error message.
- Keep Jenkins, tools and plugins updated.
- Use clear and meaningful error messages in your pipelines.
- Monitor disk space, memory and agent health.
- Document common issues and solutions for your team.
- Test fixes in a non-production environment when possible.

8. Quick Checklist

- Is Jenkins running?
- Is the agent online?
- Are required tools installed?
- Are credentials correct?
- Is there enough disk space?
- Is the network accessible?
- Is the error from your application code?



Junior Engineer Tip: Take your time to read and understand the error. Most failures tell you exactly what is wrong.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Securing a Jenkins Server

1. Security Principles

- ✓ Use least privilege access.
- ✓ Enable authentication and authorization.
- ✓ Protect sensitive data (credentials, logs, config).
- ✓ Keep Jenkins and plugins updated.
- ✓ Reduce the attack surface.
- ✓ Monitor and audit access.



2. Authentication and Authorization

- Enable authentication (do not allow anonymous access).
- Use a strong identity provider (LDAP, Active Directory, OAuth, or SSO).
- Configure role-based authorization (e.g. Role-Based Strategy).
- Grant only the minimum permissions required.
- Regularly review user accounts and permissions.



3. CSRF Protection (Concept)

- Jenkins uses CSRF protection to prevent unauthorized requests.
- Always keep CSRF protection enabled (for web UI).
- Do not disable it unless you fully understand the risk.
- Use API tokens for scripted access (not passwords).



4. Credential Security

- Store credentials in Jenkins Credential Store.
- Use the correct credential type (SSH key, token, username/password, etc.).
- Limit the scope of credentials (global or folder level).
- Use credentials binding in pipelines (do not hardcode secrets).
- Restrict who can view and use credentials.



5. Plugin Security

- Install plugins only from the official Jenkins repository.
- Keep plugins updated.
- Check security advisories for known vulnerabilities.
- Remove unused plugins.
- Avoid installing unnecessary plugins.



6. Network Exposure

- Do not expose Jenkins directly to the internet.
- Use a reverse proxy (e.g. Nginx, Apache, load balancer).
- Restrict access using firewall rules and allowlists.
- Allow access only from trusted IP ranges or VPN.
- Close unnecessary ports.



7. TLS and Reverse Proxy (Concept)

- Use HTTPS (TLS) to encrypt traffic.
- Configure a reverse proxy (Nginx/Apache).
- Terminate SSL at the proxy and forward traffic to Jenkins (internal network).
- Use a valid SSL certificate (public or internal CA).
- Redirect HTTP to HTTPS.



Reverse Proxy
(Nginx/Apache)



Jenkins
(Internal)

8. Administrator Accounts

- Create at least one admin account.
- Use a strong password and enable 2FA (if available).
- Limit the number of administrator accounts.
- Regularly review admin access.
- Use individual accounts (not shared accounts).



9. Reducing Attack Surface

- Disable unused features (e.g. CLI, remoting over TCP if not needed).
- Limit access to the Jenkins file system.
- Remove unused tools, credentials and plugins.
- Run Jenkins with a non-root user (on Linux).
- Use containerization or isolated environments where possible.
- Keep the OS and dependencies updated.



10. Best Practices

- ✓ Follow the principle of least privilege.
- ✓ Use strong authentication and authorization.
- ✓ Keep Jenkins, plugins and OS updated.
- ✓ Secure credentials and never hardcode secrets.
- ✓ Use HTTPS and a reverse proxy.
- ✓ Restrict network access.
- ✓ Regularly audit users, permissions and logs.
- ✓ Monitor for suspicious activity.



Junior Engineer Tip:

A secure Jenkins server protects your code, infrastructure and credentials. Always assume it can be a target and apply multiple layers of security.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Backing Up & Restoring Jenkins

1. What Must Be Backed Up?

- JENKINS_HOME directory (most important).
- Jenkins global configuration (system settings).
- Job configurations (Freestyle and Pipeline).
- Credentials-related data (encrypted).
- Plugins list (to reinstall the same versions).
- Security configuration (users, roles, permissions).
- Node/agent configurations (if stored in Jenkins).
- Any custom scripts, tools or configuration files.



2. Understanding JENKINS_HOME

- JENKINS_HOME contains all Jenkins data.
- Default location (Linux): /var/lib/jenkins
- Contains jobs, plugins, configs, credentials, logs, nodes, etc.
- Backing up JENKINS_HOME captures the complete state of your Jenkins instance.

```
# Check Jenkins home directory
echo $JENKINS_HOME

# Default location (on most Linux installations)
/var/lib/jenkins
```



3. Creating a Backup

- Stop Jenkins (recommended for consistency).
- Create a compressed backup of JENKINS_HOME.
- Store backups in a secure location.
- Include a backup of installed plugins list.



```
# Stop Jenkins
sudo systemctl stop jenkins

# Create a backup
sudo tar -czf jenkins-backup-$(date +%F).tar.gz
/var/lib/jenkins

# Start Jenkins
sudo systemctl start jenkins
```

4. Backup Best Practices

- ✓ Take regular automated backups (daily/weekly).
- ✓ Keep multiple backup copies (on different storage).
- ✓ Store backups outside the Jenkins server.
- ✓ Use compression to save space.
- ✓ Verify backup files are not corrupted.
- ✓ Keep plugin versions list with the backup.
- ✓ Protect backups with proper access control.
- ✓ Do not rely on a single backup location.



5. Restoring Jenkins

- Stop Jenkins before restoring.
- Replace the existing JENKINS_HOME with the backup.
- Ensure file permissions are correct.
- Start Jenkins and verify everything works.



```
# Stop Jenkins
sudo systemctl stop jenkins

# Restore from backup
sudo tar -xzf jenkins-backup-2024-01-01.tar.gz -C /

# Set correct ownership
sudo chown -R jenkins:jenkins /var/lib/jenkins

# Start Jenkins
sudo systemctl start jenkins
```

6. What to Verify After Restore

- ✓ Jenkins starts successfully.
- ✓ All jobs and pipelines are available.
- ✓ Global configuration is correct.
- ✓ Credentials and integrations work.
- ✓ Plugins are installed and compatible.
- ✓ Users, roles and permissions are intact.
- ✓ Agents/nodes are connected.
- ✓ Builds can run successfully.



7. Plugin Backup and Restore

- Save the list of installed plugins.
- During restore, install the same plugin versions.
- Avoid automatically installing the latest versions (which may cause compatibility issues).

```
# List installed plugins (Linux)
sudo cat /var/lib/jenkins/plugins.txt
```



8. Recovery Planning

- Define Recovery Time Objective (RTO) – how fast to restore.
- Define Recovery Point Objective (RPO) – how much data loss is acceptable.
- Keep a documented step-by-step restore process.
- Store backups in a different server or cloud storage.
- Regularly test the restore process (do not wait for an incident).



9. Why an Untested Backup Is Not Enough

- ✗ A backup may be corrupted or incomplete.
- ✗ You may miss critical files or configurations.
- ✗ The restore process may not work when you need it.
- ✗ Dependencies (plugins, tools, environment) may be different.
- ✗ Testing the restore process ensures your backups are usable and your recovery plan actually works.



Junior Engineer Tip:

A backup is not a backup until you have successfully restored it. Always test your backups in a non-production environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Updating & Maintaining Jenkins

1. Jenkins Releases (LTS vs Weekly)

- LTS (Long Term Support) is recommended for production environments.
- LTS receives security fixes and stability updates for a longer period.
- Weekly releases have newer features but can be less stable.
- Use LTS unless you have a specific reason to use a weekly release.

LTS
Stable
Recommended

Weekly
New Features
More Frequent
Updates

2. Java Compatibility

- Jenkins requires a compatible Java version (e.g. Java 17 or 21 for recent versions).
- Always check the Jenkins documentation for supported Java versions.
- Using an unsupported Java version can cause startup failures or unexpected issues.

```
# Check Java version
java -version
```



3. Plugin Compatibility

- Plugins must be compatible with your Jenkins version.
- Check for plugin updates before upgrading Jenkins.
- Remove unused or outdated plugins.
- Some plugins may not be immediately available for new Jenkins versions.
- Read plugin changelogs for breaking changes.



4. Pre-Upgrade Preparation

- ✓ Take a full backup of JENKINS_HOME.
- ✓ Document current Jenkins version, installed plugins and system configuration.
- ✓ Check Java compatibility.
- ✓ Review plugin compatibility.
- ✓ Identify custom configurations, scripts and integrations.



5. Staging and Testing

- Test the upgrade in a non-production (staging) environment.
- Use a copy of production data if possible.
- Verify that all jobs, pipelines, plugins and integrations work.
- Check for deprecation warnings or breaking changes.
- Fix any issues before upgrading production.



6. Maintenance Window

- Schedule the upgrade during a planned maintenance window.
- Notify users and teams in advance.
- Minimize the impact on running jobs.
- Make sure you have enough time to complete the upgrade and validation.



7. Upgrade Sequence

- 1 Take a backup of JENKINS_HOME.
- 2 Check Java version and update if required.
- 3 Update Jenkins (to the target version).
- 4 Update plugins (preferably after core upgrade).
- 5 Restart Jenkins and check for startup errors.
- 6 Verify system configuration and global settings.
- 7 Test critical jobs and pipelines.



8. Post-Upgrade Validation

- ✓ Jenkins starts successfully.
- ✓ All plugins are active and working.
- ✓ Jobs and pipelines run without errors.
- ✓ Credentials and integrations work.
- ✓ No unexpected warnings or errors in logs.
- ✓ Agents can connect and execute jobs.
- ✓ Users can access Jenkins without issues.



9. Rollback Planning

- Have a rollback plan in case the upgrade causes issues.
- Keep a copy of the previous Jenkins version (package or container image).
- Keep your JENKINS_HOME backup until the new version is stable.
- Document the rollback steps.
- Test the rollback process in staging if possible.



10. Best Practices

- ✓ Use LTS releases for production.
- ✓ Keep Jenkins, plugins and Java up to date.
- ✓ Regularly review and remove unused plugins.
- ✓ Monitor Jenkins after the upgrade.
- ✓ Keep good documentation of changes.
- ✓ Always test upgrades in a non-production environment.
- ✓ Have a clear rollback plan.



Junior Engineer Tip:

Upgrades are not just about new features. They fix security issues, improve stability and keep your Jenkins server healthy. Always plan, test and keep a backup.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Monitoring Jenkins in Production

1. Key Metrics to Monitor

- ✓ Disk usage (JENKINS_HOME and workspace).
- ✓ CPU usage.
- ✓ Memory usage (heap and non-heap).
- ✓ Queue length (pending builds).
- ✓ Executor utilization.
- ✓ Agent health (online/offline).
- ✓ Failed builds and error trends.
- ✓ Log size and growth.
- ✓ Plugin health and version status.



2. Disk Usage Monitoring

- Monitor disk usage for JENKINS_HOME and workspaces.
- Large workspaces and build artifacts can consume disk space.
- Set up alerts before the disk becomes full.
- Regularly clean old builds and artifacts.

```
# Check disk usage
df -h

# Check JENKINS_HOME size
du -sh /var/lib/jenkins

# Find large directories
du -h /var/lib/jenkins | sort -rh | head -n 10
```



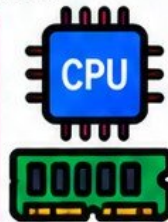
3. CPU and Memory Monitoring

- High CPU usage can indicate heavy builds or runaway tasks.
- Monitor JVM memory (heap) to avoid OutOfMemory errors.
- Track both system resources and Jenkins process.

```
# Check CPU and memory usage (Linux)
top

# Check Jenkins process
ps aux | grep jenkins

# Check JVM memory usage
jstat -gc <jenkins_pid> 5s
```

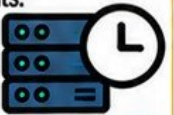


4. Queue Length and Executors

- A long queue indicates insufficient executors/agents.
- Monitor queue length and average wait time.
- Check executor utilization (busy vs idle).
- Scale up agents if the queue is consistently high.

```
# Check queue length (Jenkins CLI)
java -jar jenkins-cli.jar -s http://localhost:8080/ \
  get-queue

# Check node/executor status
java -jar jenkins-cli.jar -s http://localhost:8080/ \
  get-nodes
```



5. Agent Health Monitoring

- Ensure all required agents are online.
- Monitor agent disconnections and reconnections.
- Check agent logs for errors.
- Set up alerts for offline agents.



```
# List nodes and their status
java -jar jenkins-cli.jar -s http://localhost:8080/ \
  get-nodes

# Check agent log (example path)
tail -f /var/log/jenkins/agent.log
```

6. Failed Builds and Error Trends

- Monitor the number of failed builds over time.
- Identify common failure patterns.
- Set up alerts for a high number of failures.
- Look for specific recurring errors in console output and logs.



7. Logs and Storage Growth

- Monitor Jenkins system logs (jenkins.log).
- Track log file size and growth rate.
- Clean up old logs and rotate logs regularly.
- Check for repeated warnings or errors.



8. Alerts and Notifications

- Set up alerts for disk usage, CPU, memory, queue length.
- Alert on agent offline, high failure rate or long queue time.
- Use tools like Prometheus + Grafana, Datadog, or cloud monitoring.
- Configure email or messaging alerts (Slack, Teams, etc.).



9. Early Signs of Jenkins Instability

- Increasing queue length.
- High or unusual CPU/memory usage.
- Agents frequently going offline.
- Disk space running low.
- Many failed builds or timeouts.
- Slow UI or delayed responses.
- Repeated errors in system or agent logs.
- Growing number of plugins with warnings.



Junior Engineer Tip:

Monitor regularly and act early. Fixing small issues early prevents downtime and keeps your Jenkins server stable and reliable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Jenkins Administrator Incident Lab

1. Incident Scenario

- Builds are suddenly queuing.
- One agent is offline.**
- Disk space is running low.**
- Pipelines are beginning to fail.
- Users are reporting slow Jenkins UI.



Your task: Investigate, fix the issues, verify Jenkins is healthy and prevent recurrence.

2. Observe (Identify the Symptoms)

- Builds are waiting in the queue for a long time.
- One or more agents are offline.
- Disk space on the Jenkins server is low.
- Recent pipelines are failing.
- Jenkins UI is slow or unresponsive.



What to check:

- Jenkins dashboard (queue, agents, recent builds).
- System logs, agent logs and build console output.
- Server resources (CPU, memory, disk).
- User reports and error messages.

3. Inspect (Gather Information)

	Check queue	<code>java -jar jenkins-cli.jar -s http://localhost:8080/get-queue</code>
	Check agents	<code>java -jar jenkins-cli.jar -s http://localhost:8080/get-nodes</code>
	Check disk space	<code>df -h</code>
	Check memory/CPU	<code>top</code> (or) <code>htop</code>
	Check Jenkins logs	<code>tail -f /var/log/jenkins/jenkins.log</code>
	Check agent logs	<code>tail -f /var/log/jenkins/agent.log</code>
	Check failed builds	Check console output in Jenkins UI

4. Diagnose (Find the Root Cause)

- Agent offline:** agent service stopped, network issue, or wrong credentials.
- Disk space low:** large workspaces, build artifacts, old logs, or plugin data.
- Queued builds:** not enough executors/agents or agent offline.
- Build failures:** missing tools, permissions, credentials, application errors or environment issues.
- Slow UI:** high resource usage, large logs, many builds or disk I/O issues.



Tip: Check all possible causes. It can be more than one issue happening at the same time.

5. Fix (Resolve the Issues)

Start agent	- Check agent service and restart. - Fix network or credential issues.	
Free disk space	- Clean old workspaces, build artifacts and logs. - Remove unused files and plugins.	
Clear queue	Restart stuck agents and increase executors if needed.	
Fix build failures	Address missing tools, permissions or application errors.	
Improve performance	Monitor CPU/memory, adjust resources and clean large logs.	

6. Verify (Confirm the Fix)

- Queue is clearing and new builds start running.
- All required agents are online.
- Disk space is healthy.
- Pipelines run successfully.
- Jenkins UI is responsive.
- No new errors in system or agent logs.
- Users can trigger and run builds without issues.



7. Prevent Recurrence

- Set up monitoring and alerts (disk, CPU, memory, queue, agents).
- Implement regular log and workspace cleanup.
- Ensure enough agents/executors for workload.
- Keep backups and test recovery.
- Review and update plugins, tools and configurations regularly.
- Document what happened and the solution.



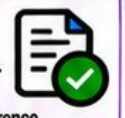
8. Useful Commands

```
# Check queue      java -jar jenkins-cli.jar -s http://localhost:8080/get-queue
# Check agents     java -jar jenkins-cli.jar -s http://localhost:8080/get-nodes
# Check disk space  df -h
# Check Jenkins logs tail -f /var/log/jenkins/jenkins.log
# Check agent logs  tail -f /var/log/jenkins/agent.log
# Clean old workspaces (example)
find /var/lib/jenkins/workspace -type d -mtime +30 -exec rm -rf {} +
```



9. Key Takeaways

- Look at the whole system, not just one error.
- Use logs, metrics and commands to find the real cause.
- Many issues are related (e.g. disk space → slow UI → build failures).
- Test your fix and make sure everything is working.
- Implement monitoring, cleanup and backups to prevent recurrence.



Junior Engineer Tip:

Incidents are normal. Stay calm, follow a structured approach and always learn from each incident to make the system more reliable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta